

Automate FW testing with MDBCore scripting, part of the MPLAB[®]X ecosystem (module3c: FW test automation with Java)

Stefan Luethin
December 2023

Agenda

1: what is the *MPLABX* ecosystem *MDBCore*, *SDK* -> how to use for automated FW testing

2: compile without IDE

3: Automated *FW-testing* with different *MDBCore* use-models

- a. simple *mdb* -> DOS/bash-script, easy to use but has limits
- b. more advanced *mdbcs* -> *.jar module needs scripting-language with Java-interface, more complex but solves *mdb* limits and opens door into Java-universe
- c. full, but most complex usage of *MDBCore* with Java -> SDK

4: summary

M3c: Short review of previous modules

- M1/Introduction in automated FW-testing with MDBCore
 - Why adding automation to FW-development and MPLABX structure
 - Terminology: three [MDBCore](#)-IFs for ***automated FW-testing***:
 - > [mdb](#) `mdb.bat` , `cmdFile`, easy to use, no install but has limits
 - > [mdbcs](#) solves `mdb`-limits, but needs scripting-language with Java-IF, big potential with any Java-lib (SwingUI)
 - > [MDBCore](#) full-blown Java, most complex but full `MDBCore`-functionality , SDK has examples
 - MPLAB^(R)X SDK: docu, NB-projects for `mdb.jar` & `mdbcs.jar` / `MDBCore.java` examples
- M2/Compile: steps to compile your FW into a debug'able *.elf?
From a FW-revision (->git) to Makefiles (-> [prjMakefilesGenerator.bat](#)) and build *.elf (-> [make](#))
- M3a/MDCore interface [mdb](#):
 - simple [MDBCore](#)-interface with `mdb.bat` = `java -jar mdb.jar`
 - commandLine interface = easy-to-use but introduces limits, no extra installation, comes with every MPLABX-version
- M3b/MDCore interface [mdbcs](#):
 - Java-module [mdbcs](#) wraps major functionality of [MDBCore](#) , which make `MDBCore` usage easy
 - Needs a scripting-language with Java-IF like Jython, JRuby or Groovy
 - Requires some setup+configuration, but solves limits of `mdb`
 - Opens door into Java-universe, like SwingUI

M3c: what is *MDBCore* ?

First let's have a brief look at *MDBCore* and where to start

- What is *MDBCore* ?
 - all functionality of MPLABX-IDE that accesses the target
 - To use *mdbcs* you don't need to be a Java-expert (just scripting-language knowledge!), whereas to use *MDBCore* with *Java*, you need good Java-knowledge
-> one reason why we created *mdbcs*
 - MPLABX-SDK is a good start for *MDBCore* -> lots of good examples
- When to use *MDBCore* ?
 - if you have your own tool and want to interface to Microchip-Debugger

M3c: preparation to use *MDBCcore*

Preparation to use MDBCcore

- you need all what is required for *mdbcs* usage
 - > so please review relevant section in module mod3b/*mdbcs*

M3b: mdb->mdbcs->mdbcore comparison

To understand the advantage of `mdbcs` over `mdb` and `MDBCore` which might be too complex, let's compare one specific function 'setBreakpoint()' in all three world

Any
scripting
knowledge

Good (!!)
Java-
knowhow

`mdb`

`mdbcs`

`MDBCore`

(dos)> mdb.bat \
mdb_cmd.txt

(dos)> groovy \
mdbcs_cmd.groovy

(dos)> netbeans
'in NB': open <sdk_mdbcore_prj>

```
1 device ATSAM21J18A
2 hwttool EDBG
3 program .\prjname.X.production.elf
4 break testEnd
5 run
6 wait 3500
7 halt
8 quit
```

```
1 package myCICDmdbcs
2 import com.microchip.mdbcs.Debugger;
3 class myTest {
4     static void main(args) {
5         new myTest().run();
6     } //end 'void main()'
7     def run() { //perform test
8         try {
9             debugger = new Debugger("ATSAMD21J18A", "EDBG", true);
10            debugger.connect(); //connect to debugger
11            debugger.loadFile("prjname.X.production.elf");
12            debugger.program(); //download elf loaded before
13            debugger.setBP("testEnd")
14            debugger.run();
15            debugger.halt();
16            while (debugger.isRunning()) {};
17            debugger.disconnect()
18            debugger = null
19            System.exit(0)
20        }
21        catch(e) {
22            println "== Ex.class= " + e.getClass();
23        }
24    } //end 'def run()'
25 } //end class myTest()
```

```
101
102 TargetAdapter targetAdapter = new MPLABXAdapter();
103 targetAdapter.setup(options); //SL: this does connect+download
104
105 System.out.println("### Using e
106 //SL: set brkpt on 'endOfTest'
107 targetAdapter.addBreakpoint("endOfTest");
```

```
Map<String, Long> fAddressMap = new HashMap<String, Long>();

private long getSymbolAddress(String symbol) {
    SymbolViewProvider symbolViewProvider = fAssembly.getLookup().lookup(
        SymbolViewProvider.class);
    Symbol sInfo = symbolViewProvider.getRawSymbol(symbol);
    if (sInfo != null) {
        long address = sInfo.Address() & 0x00000000ffffffffL;
        showInfoMsg("Address of " + symbol + " is " + address);
        return address;
    }
    return 0;
}

@Override
public boolean addBreakpoint(String value) {
    long val = getSymbolAddress(value);
    fAddressMap.put(value, val);
    if (val != 0) {
        setBreakpoint(val);
        //SL
        showInfoMsg("breakpoint set a
        return true;
    }
    return false;
}

public enum EVENTS {
    TOOL_CHANGE
}

private EVENTS {
    // compiled code
}

public Assembly()
    Compiled Code
}

public abstract String GetDirectory();
public abstract String GetDirectoryWithNoValidation();
```

Debugging

- `mdb` printf() -> ok, maybe painful
- `mdbcs` Netbeans -> state-of-art
- `MDBCore` Netbeans -> state-of-art

M3b: mdb->mdbcs->mdbcore (conclusion)

topic	mdb	mdbcs	mdbcore
installation	comes with MPLABX, no install	need Groovy, Netbeans,...	same installation requirements as for mdbcs

M3b: mdb->mdbcs->mdbcore (conclusion)

topic	mdb	mdbcs	mdbcore
installation	comes with MPLABX, no install	need Groovy, Netbeans,...	same installation requirements as for mdbcs
preparation	nothing, just use	need to setup Java-Classpath need to download+install SDK need to create mdbcs.jar	same as for mdbcs, except don't build mdbcs.jar, but run MDBCore-prj from Netbeans

M3b: mdb->mdbcs->mdbcore (conclusion)

topic	mdb	mdbcs	mdbcore
installation	comes with MPLABX, no install	need Groovy, Netbeans,...	same installation requirements as for mdbcs
preparation	nothing, just use	need to setup Java-Classpath need to download+install SDK need to create mdbcs.jar	same as for mdbcs, except don't build mdbcs.jar, but run MDBCore-prj from Netbeans
requirements	nothing, just uses simple txt-file with mdb-commands	any scripting-language with a Java-IF, like Groovy (used in this training) or Jython, Jruby (SDK has examples). Obviously can also use Java, but that increases complexity to MDBCore-level	need a development environment like Netbeans

M3b: mdb->mdbcs->mdbcore (conclusion)

topic	mdb	mdbcs	mdbcore
installation	comes with MPLABX, no install	need Groovy, Netbeans,...	same installation requirements as for mdbcs
preparation	nothing, just use	need to setup Java-Classpath need to download+install SDK need to create mdbcs.jar	same as for mdbcs, except don't build mdbcs.jar, but run MDBCore-prj from Netbeans
requirements	nothing, just uses simple txt-file with mdb-commands	any scripting-language with a Java-IF, like Groovy (used in this training) or Jython, Jruby (SDK has examples). Obviously can also use Java, but that increases complexity to MDBCore-level	need a development environment like Netbeans
knowledge, complexity	simple txt-file with some simple mdb-commands -> no specific knowledge	need some obj-oriented knowledge need to learn Groovy, BUT don't need Java- knowledge as Groovy is just like any other scripting-language.	need in-depth Java knowledge and concepts behind object-oriented programming like 'factory' or inheritance, ... -> also exposes you to low-level issues like Thread-safety, object persistence

M3b: mdb->mdbcs->mdbcore (conclusion)

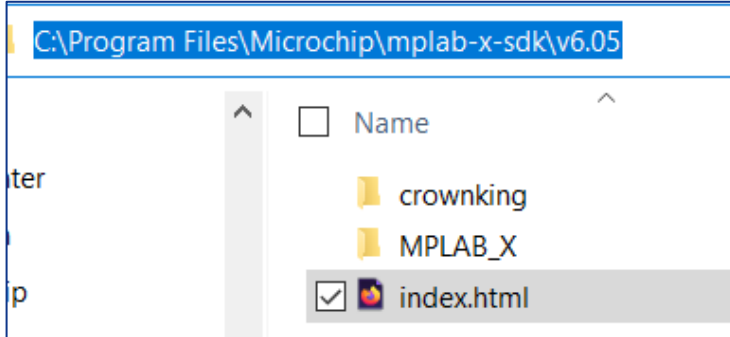
topic	mdb	mdbcs	mdbcore
installation	comes with MPLABX, no install	need Groovy, Netbeans,...	same installation requirements as for mdbcs
preparation	nothing, just use	need to setup Java-Classpath need to download+install SDK need to create mdbcs.jar	same as for mdbcs, except don't build mdbcs.jar, but run MDBCore-prj from Netbeans
requirements	nothing, just uses simple txt-file with mdb-commands	any scripting-language with a Java-IF, like Groovy (used in this training) or Jython, Jruby (SDK has examples). Obviously can also use Java, but that increases complexity to MDBCore-level	need a development environment like Netbeans
knowledge, complexity	simple txt-file with some simple mdb-commands -> no specific knowledge	need some obj-oriented knowledge need to learn Groovy, BUT don't need Java-knowledge as Groovy is just like any other scripting-language.	need in-depth Java knowledge and concepts behind object-oriented programming like 'factory' or inheritance, ... -> also exposes you to low-level issues like Thread-safety, object persistence
flexibility, functionality	only absolutely necessary functionality	very flexible and can easily be extended by importing more MDBCore-modules as needed. Wraps MDBCore-functionality to provide a much simpler interface	obviously most flexible and full MDBCore functionality

M3b: mdb->mdbcs->mdbcore (conclusion)

topic	mdb	mdbcs	mdbcore
installation	comes with MPLABX, no install	need Groovy, Netbeans,...	same installation requirements as for mdbcs
preparation	nothing, just use	need to setup Java-Classpath need to download+install SDK need to create mdbcs.jar	same as for mdbcs, except don't build mdbcs.jar, but run MDBCore-prj from Netbeans
requirements	nothing, just uses simple txt-file with mdb-commands	any scripting-language with a Java-IF, like Groovy (used in this training) or Jython, Jruby (SDK has examples). Obviously can also use Java, but that increases complexity to MDBCore-level	need a development environment like Netbeans
knowledge, complexity	simple txt-file with some simple mdb-commands -> no specific knowledge	need some obj-oriented knowledge need to learn Groovy, BUT don't need Java-knowledge as Groovy is just like any other scripting-language.	need in-depth Java knowledge and concepts behind object-oriented programming like 'factory' or inheritance, ... -> also exposes you to low-level issues like Thread-safety, object persistence
flexibility, functionality	only absolutely necessary functionality	very flexible and can easily be extended by importing more MDBCore-modules as needed. Wraps MDBCore-functionality to provide a much simpler interface	obviously most flexible and full MDBCore functionality
extention	SDK has Netbeans-prj for mdb, so can extend and build your own	like with mdb, SDK has mdbcs-Netbeans project, so can extend and build your own	obviously the full functionality as whole MDBCore-functionality available

M3c: using *MDBCore* with *Java*

A good start to use MDBCore is either the documentation



MPLAB X SDK

For MPLAB X IDE v6.05

Welcome to the MPLAB X SDK!

This software development kit contains *documentation* and *sample code* for developing:

- A **General-Purpose (GUI) plugin** for MPLAB X IDE
- A **compiler plugin** for MPLAB X IDE (integrates a compiler)
- A **tool plugin** for MPLAB X IDE (integrates a hardware debugger, programmer, or simulator)
- An **application of MDBCore** (e.g., a bench test, a command-line debugger)

The diagram on your right illustrates where each kind of software plugin/application *interfaces* with the MPLAB X Platform, as well as the *dependencies* among the largest organizational units in the MPLAB X Platform.

NOTE: Device files are no longer installed with the SDK. Currently they can be found in MPLABX under the packs directory. For example: MPLABX\v6.05/packs.

This [Source Code License and Disclaimer](#) applies to all the source files contained in this SDK.

Start Here

- [Installing and Configuring the Development Tools](#)
- [Creating a New Plugin Project](#)

Start Here

- [Installing and Configuring the Development Tools](#)
- [Creating a New Plugin Project](#)
- [Plugin User Experience Guidelines](#)
- [NetBeans Docs & Support](#) - (External link)
- [General-Purpose Plugin Tutorial](#)
- [Compiler Plugin Tutorial](#)
- [Tool Plugin Tutorial](#)
- [Signing MPLABX 3rd party plugins](#)
- [Extend Wizard Tutorial](#)
- [Simulator Programmer's Guide](#)
- [MDBCore API User's Guide](#)
- [MDBCS: Example on using MDBCore APIs](#)

Samples

- [How to Build the Samples](#)
- [MDBCore Sample "query tools"](#) - Gets the names of all connected tools
- [MDBCore Sample "connect tool"](#) - Connects to the REAL ICE
- [MDBCore Sample "program device"](#) - Programs, runs, and halts a device
- [Debug-Aware Plugin Sample](#) - A window with project/debugger awareness
- [Extend Wizard Sample](#) - Extend the new project wizard

Sources

- [Source for "mdb"](#) - The Microchip command-line debugger built upon MDBCore
- [Source for the ARM Compiler Plugin](#) - The Microchip ARM compiler plugin

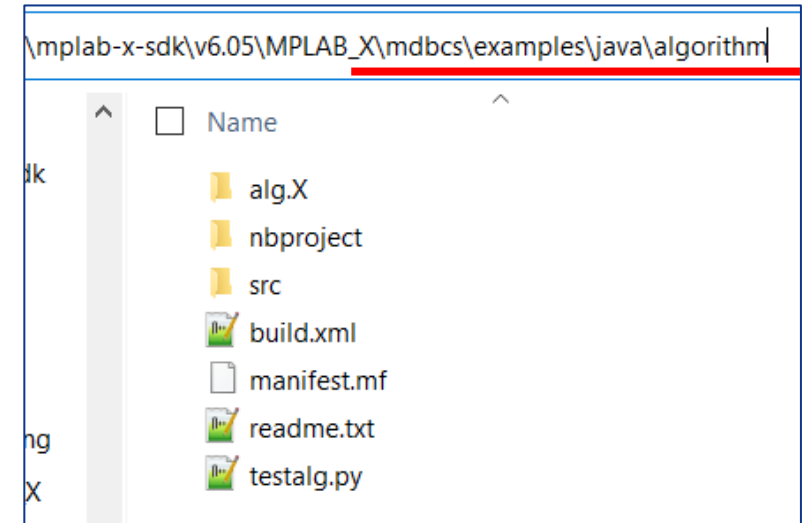
Java API References

- [mplab-ide Java API Reference](#)
- [mplab-core Java API Reference](#)
- [crownkingx Java API Reference](#)
- [MDBCS Java API Reference](#)
- [crownking_edc Java API Reference](#)
- [mplabinfo Java API Reference](#)
- [crownking_opt Java API Reference](#)
- [NetBeans v8.0.1 Java API Reference](#) - (External link)

M3b: *Java* examples using *MDBCore* in *SDK*

Examples in the SDK

- The SDK holds several interesting examples how to use MDBCore
- first one using Java is in the *mdbcs*-folder '*algorithm*' :
 - readme: explains what the example does and how to run
 - Test in Java using MDBCore-modules in Netbeans-project
 - FW as MPLABX-project



M3b: *Java* examples using *MDBCore* in *SDK*

(continued) more examples provided with the SDK

The diagram illustrates the structure of the MPLAB X SDK and its Java examples. It features a central file explorer view of the SDK directory, with several callout boxes highlighting specific components and their corresponding Java examples.

Central File Explorer View:

- Path: C:\Program Files\Microchip\mplab-x-sdk\v6.05\MPLAB_X
- Contents:
 - ☐ Name
 - doc
 - mdb
 - mdbc
 - ☒ mplab-core
 - ☒ mplab-ide
 - ☒ sample

Callout Boxes:

- platformtool** (containing ProgramFileParsing) is linked to the **sample** directory.
- debugger.mdb**, **embedded**, **embedded.makeproject**, **toolchainARM**, **toolchainXC8**, **toolchainXC16**, and **toolchainXC32** are linked to the **mplab-ide** directory.
- debug_API**, **Debug_Aware_Plugin**, **embedded_sample**, and **extendedWizardPlugin** are linked to the **mplab-core** directory.

Start Here

- [Installing and Configuring the Development Tools](#)
- [Creating a New Plugin Project](#)
- [Plugin User Experience Guidelines](#)
- [NetBeans Docs & Support](#) - (External link)
- [General-Purpose Plugin Tutorial](#)
- [Compiler Plugin Tutorial](#)
- [Tool Plugin Tutorial](#)
- [Signing MPLABX 3rd party plugins](#)
- [Extend Wizard Tutorial](#)
- [Simulator Programmer's Guide](#)
- [MDBCore API User's Guide](#)
- [MDBCS: Example on using MDBCore APIs](#)

Samples

- [How to Build the Samples](#)
- [MDBCore Sample "query_tools"](#) - Gets the names of all connected tools
- [MDBCore Sample "connect_tool"](#) - Connects to the REAL ICE
- [MDBCore Sample "program_device"](#) - Programs, runs, and halts a device
- [Debug-Aware Plugin Sample](#) - A window with project/debugger awareness
- [Extend Wizard Sample](#) - Extend the new project wizard

M3c: MDBCore - Knowledge Check

Summary and knowledge check module5: using *MDBCore*

- what is *MDBCore* ?
=> a collection of Java-module with all functionality available in the MPLABX-IDE
- knowledge level?
=> to use *MDBCore* you need good Java-knowledge, where as for *mdbcs*, knowledge of any scripting-language is enough
=> most likely *mdbcs* provides all you need, even if you want to interface to your own Java-based UI, but obviously in Java you can use all *MDBCore*-modules
- Docu and where to start
=> To start, get the SDK and review the docs mentioned in previous slides in this module
=> Or start with the Java-example in *mdbcs*-folder <mplab-x-sdk>\MPLAB_X\mdbcs\examples\java

=>so clear recommendation is to use *mdbcs*, as it

- *mdbcs* outstands the *mdb* and solves all its limits, though there is preparation needed
- provides all the advantages of a scripting language (if-else, looping, evaluating, try-catch,...)
- offers the Java-universe (SwingUIs,...)
- but still is easy enough to use and no extensive Java-knowledge is needed

End Mod3c: MDBCore

End of Module3c:

automated FW-testing using MDBCore with *Java*